SDEV 2401

Rapid Backend App Development

Class Based Views - 1

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Expectations - What I expect from you

- No Late Assignments

- No Cheating

- Be a good classmate

- Don’t waste your time

- Show up to class

WE ARE ESSENTIAL TO ALBERTA | NAIT

Agenda

On the right is what we will cover today.

- Introduction to Class Based Views
- Creating a Class Based View for a Profile Page
- Adding Class Based Views to URLs
- Refactoring a Function-Based View to a Class Based View
- Adding Decorators to Class Based Views
- Handling GET and POST in a Class Based View
- URLs for Class Based Views
- Summary

WE ARE ESSENTIAL TO ALBERTA
NAIT

Introduction to Class Based Views

- Django allows you to define views using classes, not just functions.
- Class Based Views (CBVs) help organize logic and enable code reuse.
- Useful for structuring complex web applications.
- CBVs are an alternative to function-based views.
- They make it easier to extend and maintain view logic.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Creating a Class Based View for a Profile Page

```python
from django.views import View
from django.shortcuts import render

class ProfileView(View):
    def get(self, request):
        user_profile = request.user.profile
        return render(request, 'accounts/profile.html', {'profile': user_profile})
```

- Inherit from `View` and override the `get` method.
- Fetch the user's profile and pass it to the template.
- Structure is similar to function-based views, but uses a class.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Adding Class Based Views to URLs

```
from django.urls import path
from .views import ProfileView

urlpatterns = [
    path('profile/', ProfileView.as_view(), name='profile'),
]
```

- Use `.as_view()` to convert the class into a callable view for URLs.
- This is required for all CBVs in Django.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Refactoring a Function-Based View to a Class Based View

```
from django.views import View
from django.shortcuts import render
from .models import BlogPost

class BlogListView(View):
    template_name = 'blog/list.html'

    def get(self, request):
        posts = BlogPost.objects.filter(published=True)
        return render(request, self.template_name, {'posts': posts})
```

- Move logic from a function-based view into the `get` method of a class.
- Use attributes like `template_name` for clarity and reuse.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Adding Decorators to Class Based Views

```
from django.utils.decorators import method_decorator
from django.contrib.auth.decorators import login_required

@method_decorator(login_required, name='dispatch')
class BlogListView(View):
    # ... existing code ...
```

- Use `method_decorator` to apply decorators (like `login_required`) to CBVs.
- Decorators are applied to the `dispatch` method, affecting all request types.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Handling GET and POST in a Class Based View

```
from django.views import View
from django.utils.decorators import method_decorator
from django.contrib.auth.decorators import login_required
from .forms import FeedbackForm

@method_decorator(login_required, name='dispatch')
class FeedbackView(View):
    template_name = 'feedback/submit.html'
    form_class = FeedbackForm

    def get(self, request):
        form = self.form_class()
        return render(request, self.template_name, {'form': form})

    def post(self, request):
        form = self.form_class(request.POST)
        if form.is_valid():
            form.save()
            return redirect('feedback_thankyou')
        return render(request, self.template_name, {'form': form})
```

- Override both `get` and `post` methods to handle different request types.
- Use decorators for authentication.

URLs for Class Based Views

```
from django.urls import path
from .views import BlogListView, FeedbackView

urlpatterns = [
    path('blog/', BlogListView.as_view(), name='blog_list'),
    path('feedback/', FeedbackView.as_view(), name='feedback_submit'),
]
```

- Update URL patterns to use `.as_view()` for each CBV.
- This enables Django to route requests to your class-based views.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Summary

- Class Based Views (CBVs) provide a structured way to define views in Django.
- They allow for better organization and code reuse, though this is debated in the community.
- CBVs can handle multiple HTTP methods and can be decorated for authentication and other purposes.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Example

Let’s go do an example together